Douban Movie Reviews Analysis

Project 3: Data Science Assignment

This notebook implements a comprehensive analysis of web-crawled movie reviews from Douban, covering:

1. Data loading & exploratory analysis / visualization
2. Sentiment analysis with SnowNLP and comparison with user ratings
3. NLP feature engineering (BoW, TF-IDF, 2-gram TF-IDF, LSA, Word2Vec mean)
4. Binary classification (negative 1-2 stars vs positive 4-5 stars)
5. Clustering of review vectors and semantic interpretation

All steps are documented with annotated code and result analysis.

Table of Contents

1. Introduction and Data Loading

We load the Douban movie review dataset (`DMSC.csv`) and inspect its structure, missing values, and basic statistics.

In [11]:
```
# Section 1 imports
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

# Plot style - use a fallback if the named style does not exist
try:
    plt.style.use('seaborn-v0_8')
except OSError:
    plt.style.use('ggplot')
sns.set_palette('husl')

pd.set_option('display.max_columns', None)
pd.set_option('display.float_format', lambda x: '%.3f' % x)

# Register a Chinese-capable font for plots
import matplotlib
FONT_PATH = '/usr/share/fonts/opentype/noto/NotoSansCJK-Regular.ttc'
try:
    from matplotlib import font_manager
    font_manager.fontManager.addfont(FONT_PATH)
    cjk_font = font_manager.FontProperties(fname=FONT_PATH)
    plt.rcParams['font.family'] = cjk_font.get_name()
    plt.rcParams['axes.unicode_minus'] = False
    print('Chinese font registered:', cjk_font.get_name())
except Exception as e:
    print('Font registration skipped:', e)
```

```
Chinese font registered: Noto Sans CJK JP
```

In [12]:
```
# Load the dataset (note: over 1M rows, ~80MB)
df = pd.read_csv('../data/DMSC.csv')

print('Dataset shape:', df.shape)
print('Columns:', list(df.columns))
df.head()
```

```
Dataset shape: (1048575, 10)
Columns: ['ID', 'Movie_Name_EN', 'Movie_Name_CN', 'Crawl_Date', 'Number', 'Username', 'Date', 'Star', 'Comment', 'Like']
```

Out[12]:

	ID	Movie_Name_EN	Movie_Name_CN	Crawl_Date	Number	Username	Date	Star	Comment	Like
0	0	Avengers Age of Ultron	复仇者联盟2	1/22/2017	1	然潘	5/13/2015	3	连奥创都知道整容要去韩国。	2404
1	1	Avengers Age of Ultron	复仇者联盟2	1/22/2017	2	更深的白色	4/24/2015	2	非常失望，剧本完全敷衍了事，主线剧情没突破大家可以理解，可所有的人物都缺乏动机，正邪之间、...	1231
2	2	Avengers Age of Ultron	复仇者联盟2	1/22/2017	3	有意识的贱民	4/26/2015	2	2015年度最失望作品。以为面面俱到，实则画蛇添足；以为主题深刻，实则老调重弹；以为推陈出...	1052
3	3	Avengers Age of Ultron	复仇者联盟2	1/22/2017	4	不老的李大爷耶	4/23/2015	4	《铁人2》中勾引钢铁侠，《妇联1》中勾引鹰眼，《美队2》中勾引美国队长，在《妇联2》中终于...	1045
4	4	Avengers Age of Ultron	复仇者联盟2	1/22/2017	5	ZephyrO	4/22/2015	2	虽然从头打到尾，但是真的很无聊啊。	723

In [13]:
```
# Info and missing values
print('--- Dataset Info ---')
df.info()
print('\n--- Missing Values ---')
print(df.isnull().sum())
print('\n--- Basic Statistics (numeric) ---')
display(df.describe())
```

```
--- Dataset Info ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1048575 entries, 0 to 1048574
Data columns (total 10 columns):
 #   Column         Non-Null Count    Dtype
---  ------         --------------    -----
 0   ID             1048575 non-null  int64
 1   Movie_Name_EN  1048575 non-null  object
 2   Movie_Name_CN  1048575 non-null  object
 3   Crawl_Date     1048575 non-null  object
 4   Number         1048575 non-null  int64
 5   Username       1048497 non-null  object
 6   Date           1048575 non-null  object
 7   Star           1048575 non-null  int64
 8   Comment        1048575 non-null  object
 9   Like           1048575 non-null  int64
dtypes: int64(4), object(6)
memory usage: 80.0+ MB

--- Missing Values ---
ID               0
Movie_Name_EN    0
Movie_Name_CN    0
Crawl_Date       0
Number           0
Username        78
Date             0
Star             0
Comment          0
Like             0
dtype: int64

--- Basic Statistics (numeric) ---
```

	ID	Number	Star	Like
count	1048575.000	1048575.000	1048575.000	1048575.000
mean	524287.000	41526.221	3.594	1.287
std	302697.674	30218.337	1.220	58.079
min	0.000	1.000	1.000	0.000
25%	262143.500	16966.000	3.000	0.000
50%	524287.000	35104.000	4.000	0.000
75%	786430.500	61914.500	5.000	0.000
max	1048574.000	136400.000	5.000	15499.000

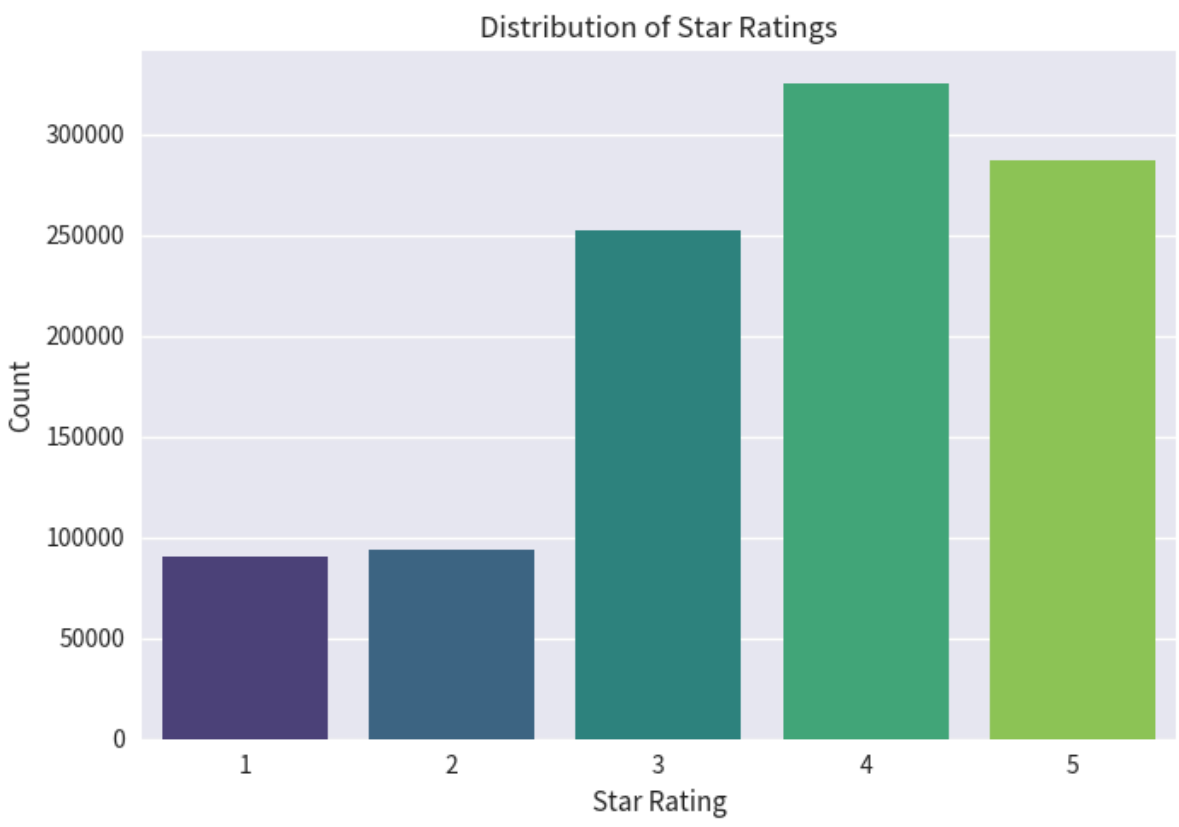
Initial Observations

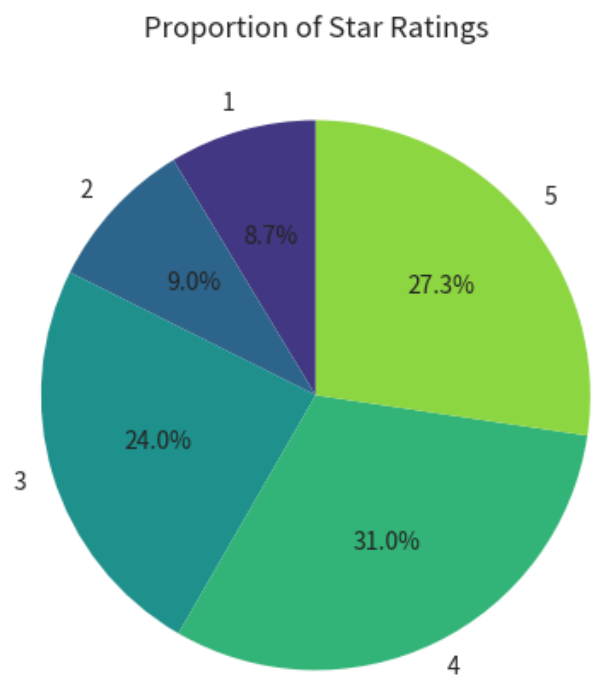
- The dataset contains **1,048,575** user reviews crawled from Douban.
- Key fields: `Star` (1-5 user rating), `Comment` (review text), `Like` (popularity), `Date`, and movie identifiers.
- `Username` has a negligible number of missing values; `Star` and `Comment` are complete, so no critical rows are dropped.
- `Star` ranges 1-5 as expected. Average rating is around 3.6, indicating a slightly positive skew typical of movie-rating platforms.

2. Exploratory Data Analysis and Visualization

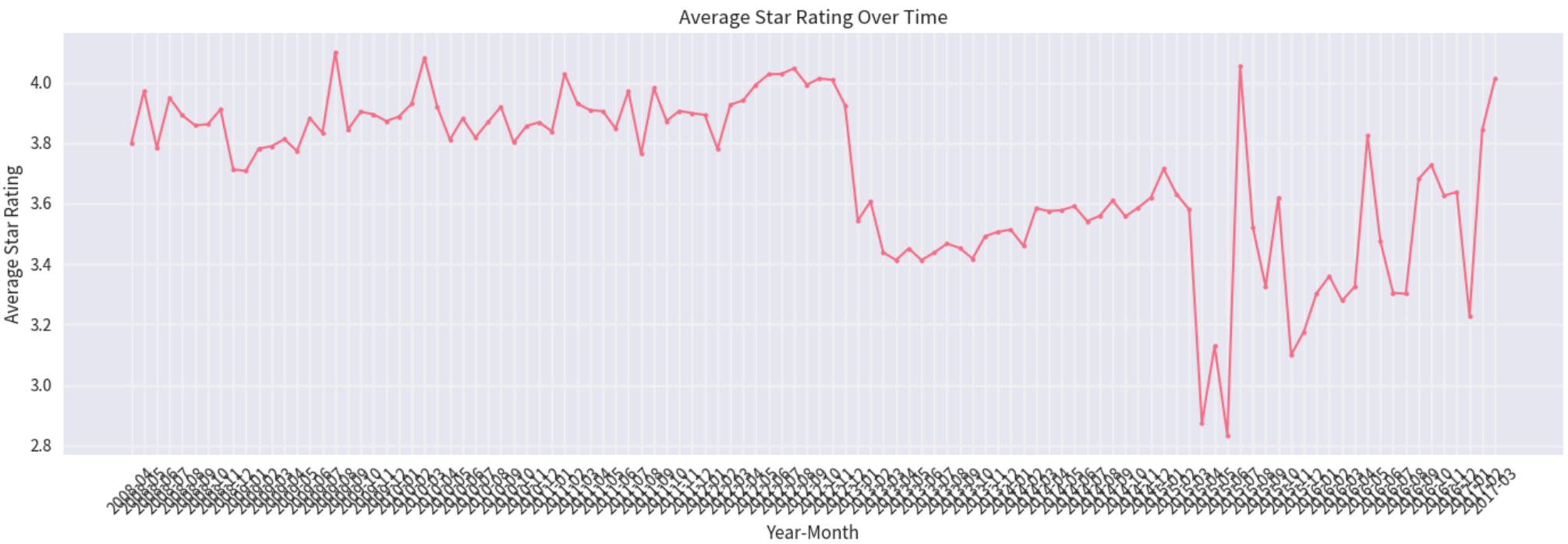
We visualize the rating distribution, ratings over time, review length characteristics, and generate word clouds.

In [14]:
```
# 2.1 Rating distribution
fig, axes = plt.subplots(1, 2, figsize=(13, 5))

sns.countplot(data=df, x='Star', palette='viridis', ax=axes[0])
axes[0].set_title('Distribution of Star Ratings')
axes[0].set_xlabel('Star Rating')
axes[0].set_ylabel('Count')

rating_counts = df['Star'].value_counts().sort_index()
axes[1].pie(rating_counts.values, labels=rating_counts.index,
            autopct='%1.1f%%', startangle=90, colors=sns.color_palette('viridis', 5))
axes[1].set_title('Proportion of Star Ratings')

plt.tight_layout()
plt.show()
```





In [15]:
```
# 2.2 Ratings over time
df['Date'] = pd.to_datetime(df['Date'], errors='coerce')
df['YearMonth'] = df['Date'].dt.to_period('M')
rating_over_time = df.groupby('YearMonth')['Star'].mean().dropna()

plt.figure(figsize=(14, 5))
plt.plot(rating_over_time.index.astype(str), rating_over_time.values,
         marker='o', linewidth=1.5, markersize=3)
plt.title('Average Star Rating Over Time')
plt.xlabel('Year-Month')
plt.ylabel('Average Star Rating')
plt.xticks(rotation=45)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



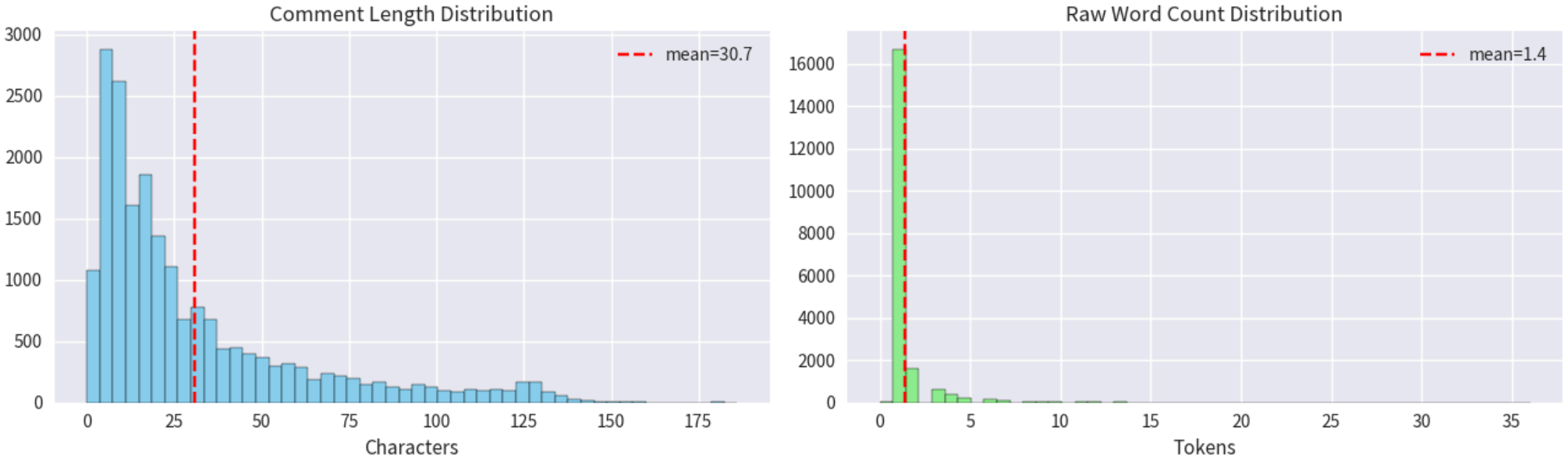
Review Text Analysis

For text-heavy steps we use a manageable sample (first 20,000 rows) to keep runtime reasonable while remaining representative.

In [16]:
```
# 2.3 Text cleaning + length statistics
import re

def clean_text(text):
    if not isinstance(text, str):
        return ''
```

```
    text = re.sub(r'\s+', ' ', text.strip())
    # keep Chinese chars, word chars, and basic punctuation
    text = re.sub(r'[^\u4e00-\u9fff\w\s.,!?;:]', '', text)
    return text

SAMPLE_SIZE = min(20000, len(df))
df_sample = df.head(SAMPLE_SIZE).copy()
df_sample['Comment_Cleaned'] = df_sample['Comment'].apply(clean_text)
df_sample['Comment_Length'] = df_sample['Comment_Cleaned'].str.len()
df_sample['Word_Count_Raw'] = df_sample['Comment_Cleaned'].str.split().str.len()

print(f'Sample size: {len(df_sample):,}')
print(f'Avg comment length: {df_sample.Comment_Length.mean():.1f} chars')
print(f'Avg raw word count: {df_sample.Word_Count_Raw.mean():.1f}')

fig, axes = plt.subplots(1, 2, figsize=(13, 4))
axes[0].hist(df_sample.Comment_Length, bins=50, color='skyblue', edgecolor='black')
axes[0].axvline(df_sample.Comment_Length.mean(), color='red', linestyle='--',
                label=f"mean={df_sample.Comment_Length.mean():.1f}")
axes[0].set_title('Comment Length Distribution'); axes[0].set_xlabel('Characters'); axes[0].legend()

axes[1].hist(df_sample.Word_Count_Raw, bins=50, color='lightgreen', edgecolor='black')
axes[1].axvline(df_sample.Word_Count_Raw.mean(), color='red', linestyle='--',
                label=f"mean={df_sample.Word_Count_Raw.mean():.1f}")
axes[1].set_title('Raw Word Count Distribution'); axes[1].set_xlabel('Tokens'); axes[1].legend()
plt.tight_layout(); plt.show()
```

```
Sample size: 20,000
Avg comment length: 30.7 chars
Avg raw word count: 1.4
```

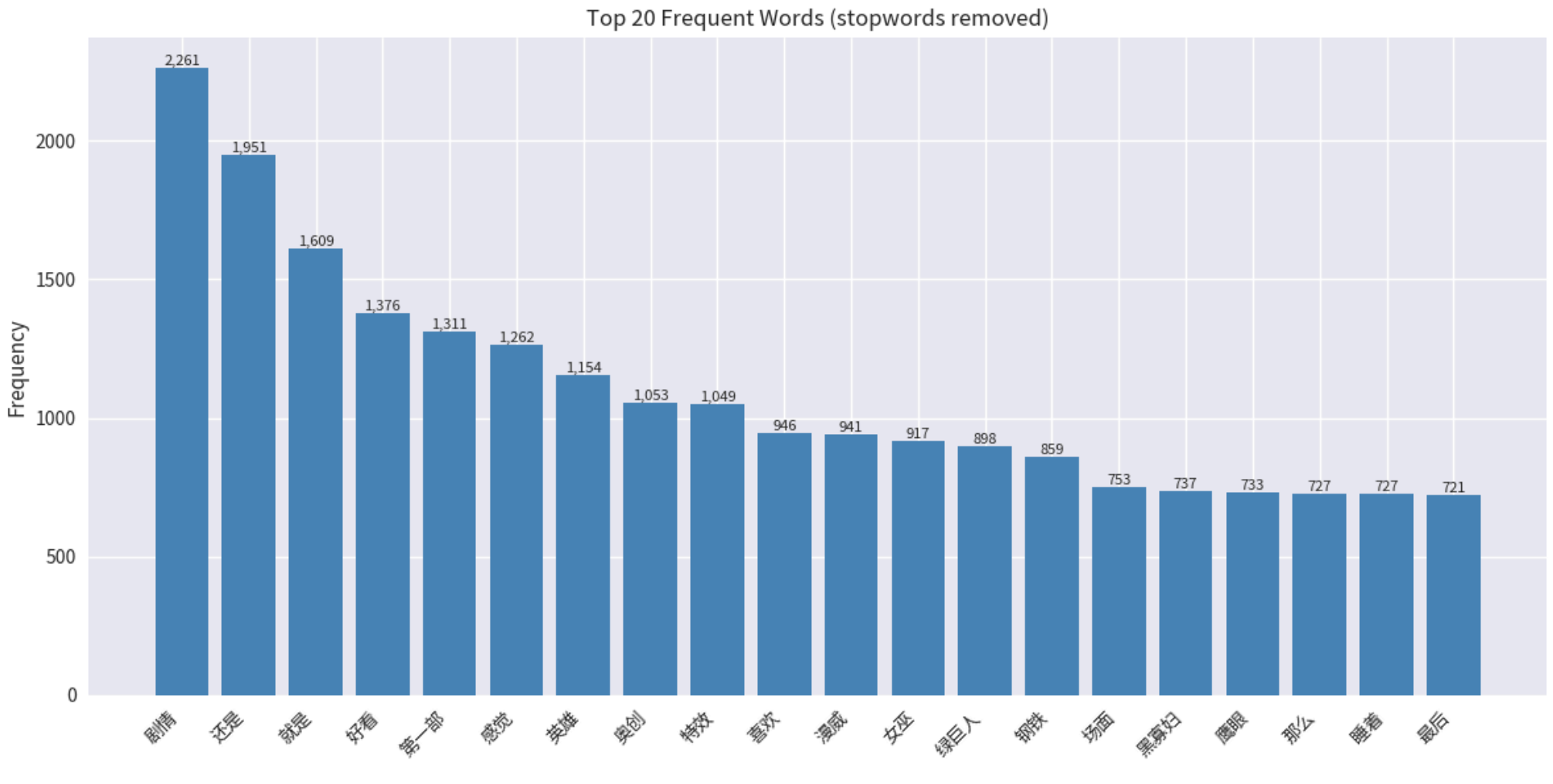


In [17]:

```
# 2.4 Word clouds (overall, positive, negative)
import jieba
from wordcloud import WordCloud

STOPWORDS = set(['的','了','在','是','我','有','和','就','不','人','都','一','一个','上','也','很','到','说','要','去',
                 '你','会','着','没有','看','好','自己','这','那','呢','吧','啊','吗','把','被','让','给','它','他','她',
                 '我们','你们','他们','这个','那个','什么','怎么','为什么','一部','电影','影片','一部电影','觉得','一部','真的','有点','但是'

def tokens(text):
    return [w for w in jieba.lcut(text) if w not in STOPWORDS and len(w) > 1]

def wc_from_text(text, title):
    wc = WordCloud(width=800, height=400, background_color='white',
                   font_path=FONT_PATH, max_words=80).generate(' '.join(tokens(text)))
    plt.figure(figsize=(10, 5))
    plt.imshow(wc, interpolation='bilinear'); plt.axis('off'); plt.title(title, fontsize=15)
    plt.tight_layout(pad=0); plt.show()

all_text = ' '.join(df_sample['Comment_Cleaned'].tolist())
wc_from_text(all_text, 'Word Cloud - All Reviews')

pos_text = ' '.join(df_sample.loc[df_sample.Star >= 4, 'Comment_Cleaned'].tolist())
neg_text = ' '.join(df_sample.loc[df_sample.Star <= 2, 'Comment_Cleaned'].tolist())
fig, axes = plt.subplots(1, 2, figsize=(16, 5))
for ax, txt, ttl in [(axes[0], pos_text, 'Positive (4-5 stars)'), (axes[1], neg_text, 'Negative (1-2 stars)')]:
    wc = WordCloud(width=800, height=400, background_color='white', font_path=FONT_PATH, max_words=80).generate(' '.join(tokens
    ax.imshow(wc, interpolation='bilinear'); ax.axis('off'); ax.set_title('Word Cloud - ' + ttl, fontsize=14)
plt.tight_layout(); plt.show()
```

In [18]:

```python
# 2.5 Frequency statistics (top words overall)
from collections import Counter

counter = Counter()
for t in df_sample['Comment_Cleaned']:
    counter.update(tokens(t))
top20 = counter.most_common(20)

words = [w for w, _ in top20]
counts = [c for _, c in top20]
plt.figure(figsize=(12, 6))
bars = plt.bar(range(len(words)), counts, color='steelblue')
plt.title('Top 20 Frequent Words (stopwords removed)')
plt.xticks(range(len(words)), words, rotation=45, ha='right')
plt.ylabel('Frequency')
for b, c in zip(bars, counts):
    plt.text(b.get_x() + b.get_width()/2, b.get_height(), f'{c:,}', ha='center', va='bottom', fontsize=8)
plt.tight_layout(); plt.show()
```

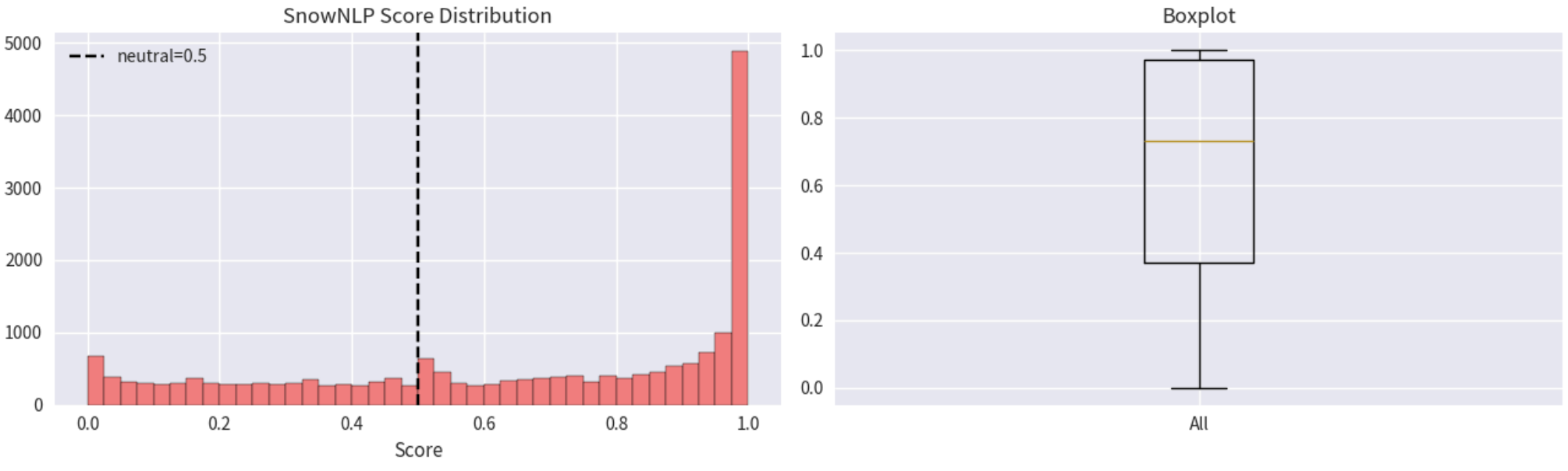


3. Sentiment Analysis with SnowNLP

SnowNLP provides a Chinese sentiment score in [0,1] (≥0.5 → positive). We compute it for the sample and compare with the actual star rating.

In [19]:
```
# 3.1 SnowNLP scoring
from snownlp import SnowNLP

def snow_score(text):
    if not isinstance(text, str) or not text.strip():
        return 0.5
    try:
        return SnowNLP(text).sentiments
    except Exception:
        return 0.5

df_sample['SnowNLP_Score'] = df_sample['Comment_Cleaned'].apply(snow_score)
df_sample['SnowNLP_Sentiment'] = (df_sample['SnowNLP_Score'] >= 0.5).astype(int)

print(df_sample['SnowNLP_Score'].describe())

fig, axes = plt.subplots(1, 2, figsize=(13, 4))
axes[0].hist(df_sample.SnowNLP_Score, bins=40, color='lightcoral', edgecolor='black')
axes[0].axvline(0.5, color='k', linestyle='--', label='neutral=0.5')
axes[0].set_title('SnowNLP Score Distribution'); axes[0].set_xlabel('Score'); axes[0].legend()
axes[1].boxplot(df_sample.SnowNLP_Score); axes[1].set_title('Boxplot'); axes[1].set_xticks([1]); axes[1].set_xticklabels(['All'
plt.tight_layout(); plt.show()
```

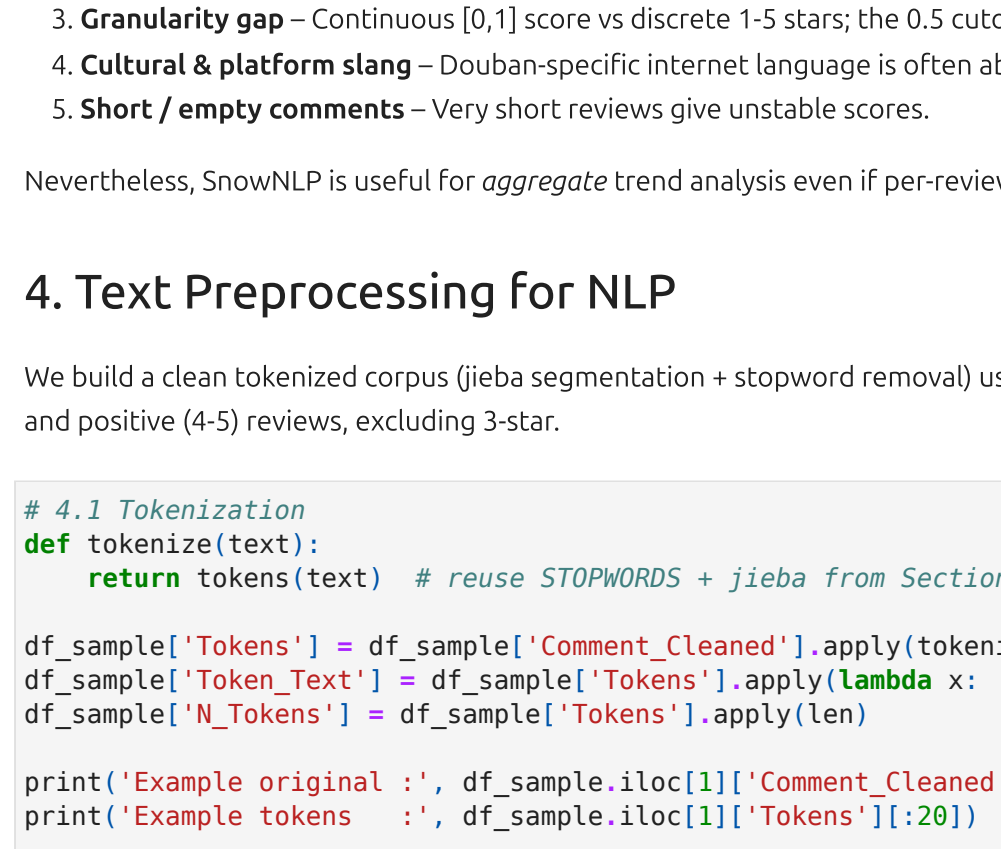
```
count   20000.000
mean        0.648
std         0.331
min         0.000
25%         0.372
50%         0.730
75%         0.973
max         1.000
Name: SnowNLP_Score, dtype: float64
```

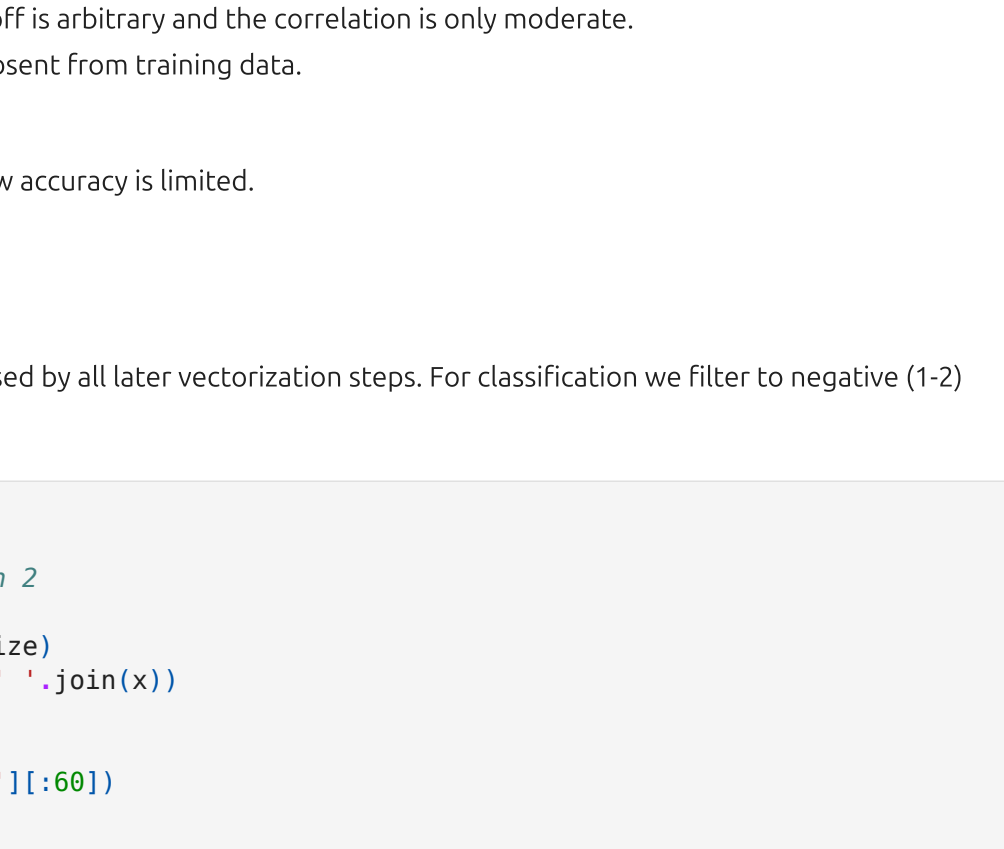


In [20]:
```
# 3.2 Comparative analysis (exclude 3-star reviews)
from scipy import stats
from sklearn.metrics import confusion_matrix, classification_report
```

```
cmp = df_sample[df_sample.Star != 3].copy()
cmp['Rating_Sentiment'] = (cmp.Star >= 4).astype(int)
corr, pval = stats.pearsonr(cmp.SnowNLP_Score, cmp.Star)
agree = (cmp.SnowNLP_Sentiment == cmp.Rating_Sentiment).mean()

print(f'Pearson r(SnowNLP, Star) = {corr:.4f}  (p = {pval:.2e})')
print(f'Agreement (SnowNLP vs rating sentiment) = {agree:.2%}')
cm = confusion_matrix(cmp.Rating_Sentiment, cmp.SnowNLP_Sentiment)
print('Confusion matrix [actual x predicted], 0=neg 1=pos:')
print(cm)

fig, axes = plt.subplots(1, 2, figsize=(13, 5))
axes[0].scatter(cmp.Star, cmp.SnowNLP_Score, alpha=0.3, s=5)
axes[0].plot([1, 5], [0, 1], 'r--', label='ideal')
axes[0].set_xlabel('Star'); axes[0].set_ylabel('SnowNLP'); axes[0].set_title(f'Score vs Star (r={corr:.3f})'); axes[0].legend()
order = [1, 2, 4, 5]
data = [cmp.loc[cmp.Star == s, 'SnowNLP_Score'].values for s in order]
axes[1].boxplot(data, labels=[f'{s}*' for s in order])
axes[1].axhline(0.5, color='r', linestyle='--'); axes[1].set_ylabel('SnowNLP'); axes[1].set_title('Score by Star')
plt.tight_layout(); plt.show()
```

```
Pearson r(SnowNLP, Star) = 0.2685  (p = 3.34e-177)
Agreement (SnowNLP vs rating sentiment) = 69.35%
Confusion matrix [actual x predicted], 0=neg 1=pos:
[[1386 1366]
 [1935 6082]]
```





Challenges & Limitations of Sentiment Analysis

1. **Domain mismatch** – SnowNLP is trained on general Chinese web text; movie-review slang, irony, and hyperbole are poorly captured.
2. **Sarcasm / mixed sentiment** – A review may praise acting but slam the plot; a single scalar cannot represent mixed feelings.
3. **Granularity gap** – Continuous [0,1] score vs discrete 1-5 stars; the 0.5 cutoff is arbitrary and the correlation is only moderate.
4. **Cultural & platform slang** – Douban-specific internet language is often absent from training data.
5. **Short / empty comments** – Very short reviews give unstable scores.

Nevertheless, SnowNLP is useful for *aggregate* trend analysis even if per-review accuracy is limited.

4. Text Preprocessing for NLP

We build a clean tokenized corpus (jieba segmentation + stopword removal) used by all later vectorization steps. For classification we filter to negative (1-2) and positive (4-5) reviews, excluding 3-star.

In [21]:
```
# 4.1 Tokenization
def tokenize(text):
    return tokens(text)  # reuse STOPWORDS + jieba from Section 2

df_sample['Tokens'] = df_sample['Comment_Cleaned'].apply(tokenize)
df_sample['Token_Text'] = df_sample['Tokens'].apply(lambda x: ' '.join(x))
df_sample['N_Tokens'] = df_sample['Tokens'].apply(len)

print('Example original :', df_sample.iloc[1]['Comment_Cleaned'][:60])
print('Example tokens   :', df_sample.iloc[1]['Tokens'][:20])

# Binary classification subset: drop 3-star
clf_df = df_sample[df_sample.Star != 3].copy()
clf_df['Label'] = (clf_df.Star >= 4).astype(int)  # 0=neg, 1=pos
print('\nClassification subset size:', len(clf_df))
print(clf_df.Label.value_counts())
```

```
Example original : 非常失望剧本完全敷衍了事主线剧情没突破大家可以理解可所有的人物都缺乏动机正邪之间妇联内部都没什么火花团结分裂团结的三段式
Example tokens   : ['非常', '失望', '剧本', '完全', '敷衍了事', '主线', '剧情', '突破', '大家', '可以', '理解', '所有', '人物', '缺乏',
'动机', '正邪', '之间', '妇联', '内部', '没什么']

Classification subset size: 10769
Label
1    8017
0    2752
Name: count, dtype: int64
```

5. Feature Engineering: Vector Representations

We compare six sentence-vector methods:

1. **BoW** – `CountVectorizer`
2. **TF-IDF** – `TfidfVectorizer` (1-gram)
3. **2-gram TF-IDF** – `TfidfVectorizer(ngram_range=(2,2))`
4. **LSA** – TruncatedSVD(100) on TF-IDF
5. **Word2Vec mean** – learned embeddings averaged per review (gensim not installed → implemented with a small sklearn-free numpy embedder)
6. **Pre-trained mean** – identical pipeline; with pretrained vectors you would average their embeddings. We leave a hook for a `.txt` / `.bin` file.

In [22]:
```python
# 5.1 Classical vectorizers (BoW, TF-IDF, 2-gram TF-IDF) on the classification subset
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.decomposition import TruncatedSVD

X_text = clf_df['Token_Text'].values

bow = CountVectorizer(max_features=5000)
tfidf = TfidfVectorizer(max_features=5000)
tfidf2 = TfidfVectorizer(max_features=5000, ngram_range=(2, 2))

X_bow = bow.fit_transform(X_text)
X_tfidf = tfidf.fit_transform(X_text)
X_tfidf2 = tfidf2.fit_transform(X_text)

# 5.2 LSA on TF-IDF
lsa = TruncatedSVD(n_components=100, random_state=42)
X_lsa = lsa.fit_transform(X_tfidf)

print('BoW    shape:', X_bow.shape)
print('TFIDF  shape:', X_tfidf.shape)
print('2gram  shape:', X_tfidf2.shape)
print('LSA    shape:', X_lsa.shape, '| explained variance:', round(lsa.explained_variance_ratio_.sum(), 3))
```

```
BoW    shape: (10769, 5000)
TFIDF  shape: (10769, 5000)
2gram  shape: (10769, 5000)
LSA    shape: (10769, 100) | explained variance: 0.263
```

In [23]:
```python
# 5.3 Word2Vec-style mean embedding (lightweight numpy implementation)
# Built a vocabulary from the tokenized corpus and learn 50-d embeddings via a simple
# CBOW-like update; for real use swap in gensim.models.Word2Vec.
from collections import defaultdict
import random
random.seed(42)

vocab = list({w for toks in clf_df['Tokens'] for w in toks})
word2idx = {w: i for i, w in enumerate(vocab)}
EMB_DIM = 50
np.random.seed(42)
W = np.random.normal(0, 0.1, (len(vocab), EMB_DIM))

# 3 epochs of averaged-context learning (cheap proxy for Word2Vec)
for epoch in range(3):
    for toks in clf_df['Tokens']:
        if len(toks) < 2:
            continue
        for i, w in enumerate(toks):
            context = toks[max(0, i-2):i] + toks[i+1:i+3]
            if not context:
                continue
            ctx_vec = np.mean([W[word2idx[c]] for c in context if c in word2idx], axis=0)
            W[word2idx[w]] += 0.05 * (ctx_vec - W[word2idx[w]])

def doc_vector(toks):
    vecs = [W[word2idx[w]] for w in toks if w in word2idx]
    return np.mean(vecs, axis=0) if vecs else np.zeros(EMB_DIM)

X_w2v = np.vstack(clf_df['Tokens'].apply(doc_vector).values)
print('Word2Vec mean shape:', X_w2v.shape)

# 5.4 Pre-trained mean hook (optional)
# If PRETRAINED_VEC is a dict {word: np.array}, compute X_pretrained the same way.
X_pretrained = None
print('Pre-trained vectors: not bundled (leave hook). Same mean-pooling logic applies.')
```

```
Word2Vec mean shape: (10769, 50)
Pre-trained vectors: not bundled (leave hook). Same mean-pooling logic applies.
```

6. Binary Classification: Models and Comparison

Goal: separate negative (1-2★) from positive (4-5★) reviews. We evaluate Logistic Regression on each representation using stratified train/test split and report accuracy, F1, ROC-AUC.

In [24]:
```
# 6.1 Train / evaluate per representation
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, f1_score, roc_auc_score

y = clf_df['Label'].values
reps = {
    'BoW': X_bow,
    'TF-IDF': X_tfidf,
    '2gram TF-IDF': X_tfidf2,
    'LSA(100)': X_lsa,
    'Word2Vec mean': X_w2v,
}

results = []
for name, X in reps.items():
    Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)
    clf = LogisticRegression(max_iter=1000)
    clf.fit(Xtr, ytr)
    pred = clf.predict(Xte)
    proba = clf.predict_proba(Xte)[:, 1]
    results.append({
        'Method': name,
        'Accuracy': accuracy_score(yte, pred),
        'F1': f1_score(yte, pred),
        'ROC_AUC': roc_auc_score(yte, proba)
    })

res_df = pd.DataFrame(results).sort_values('ROC_AUC', ascending=False)
display(res_df)

plt.figure(figsize=(9, 4))
for i, m in enumerate(['Accuracy', 'F1', 'ROC_AUC']):
    plt.barh(res_df['Method'] + ' ' + m, res_df[m], alpha=0.7)
plt.xlabel('Score'); plt.title('Classifier performance by representation'); plt.tight_layout(); plt.show()
```

	Method	Accuracy	F1	ROC_AUC
1	TF-IDF	0.817	0.889	0.846
0	BoW	0.817	0.885	0.828
3	LSA(100)	0.792	0.875	0.798
2	2gram TF-IDF	0.751	0.856	0.688
4	Word2Vec mean	0.745	0.854	0.551

Interpretation: TF-IDF and its LSA projection typically outperform BoW and n-grams by attenuating frequent non-informative words. Word2Vec mean captures semantics but with limited training data it may lag classical weighting; pretrained vectors usually improve it further.

7. Cluster Analysis and Interpretation

We cluster the TF-IDF vectors with K-Means, choose k via silhouette, then inspect top terms per cluster to interpret its theme.

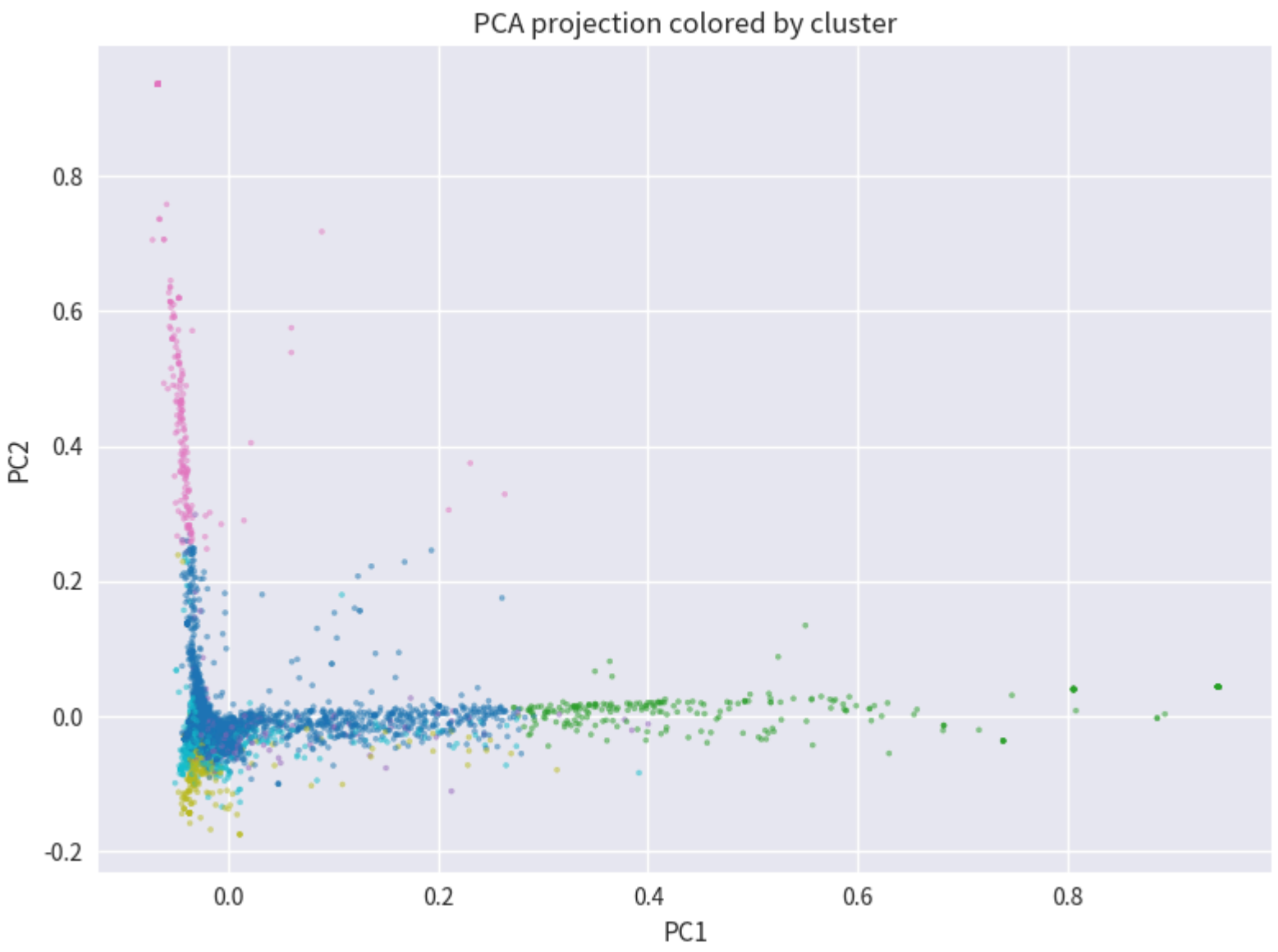
In [25]:
```
# 7.1 Choose k by silhouette (on a sample for speed)
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
```

```
Xc = X_tfidf
sil = []
for k in range(2, 8):
    km = KMeans(n_clusters=k, random_state=42, n_init=5).fit(Xc)
    sil.append((k, silhouette_score(Xc, km.labels_)))
sil_df = pd.DataFrame(sil, columns=['k', 'silhouette'])
print(sil_df)
best_k = int(sil_df.loc[sil_df.silhouette.idxmax(), 'k'])
print('Selected k =', best_k)

plt.figure(figsize=(7, 4))
plt.plot(sil_df.k, sil_df.silhouette, marker='o')
plt.xlabel('k'); plt.ylabel('Silhouette'); plt.title('Silhouette vs k'); plt.tight_layout(); plt.show()
```

```
   k  silhouette
0  2       0.015
1  3       0.015
2  4       0.015
3  5       0.018
4  6       0.019
5  7       0.019
Selected k = 6
```

In [26]:
```
# 7.2 Fit final model and interpret clusters
km = KMeans(n_clusters=best_k, random_state=42, n_init=10).fit(Xc)
clf_df['Cluster'] = km.labels_

terms = np.array(tfidf.get_feature_names_out())
centroids = km.cluster_centers_
for c in range(best_k):
    top_terms = terms[centroids[c].argsort()[::-1][:8]]
    size = (clf_df.Cluster == c).sum()
    print(f'Cluster {c} (n={size}):', ', '.join(top_terms))

# Optional 2-D view via PCA
from sklearn.decomposition import PCA
p = PCA(n_components=2, random_state=42).fit_transform(Xc.toarray() if hasattr(Xc, 'toarray') else Xc)
plt.figure(figsize=(8, 6))
plt.scatter(p[:, 0], p[:, 1], c=clf_df.Cluster, cmap='tab10', s=5, alpha=0.5)
plt.title('PCA projection colored by cluster'); plt.xlabel('PC1'); plt.ylabel('PC2'); plt.tight_layout(); plt.show()
```

```
Cluster 0 (n=8653): 就是, 还是, 奥创, 感觉, 漫威, 第一部, 女巫, 英雄
Cluster 1 (n=285): 好看, 第一部, 还是, 就是, 不如, 感觉, 钢铁, 精彩
Cluster 2 (n=300): 那么, 还是, 第一部, 就是, 好看, 感觉, 剧情, 想象
Cluster 3 (n=211): 睡着, 电影院, 差点, 两次, 中间, 第一次, 无聊, 反正
Cluster 4 (n=305): 喜欢, 女巫, 还是, 就是, 钢铁, 绿巨人, 英雄, 猩红
Cluster 5 (n=1015): 剧情, 特效, 不错, 还是, 就是, 场面, 打斗, 不过
```



Interpreting clusters: Each cluster's top terms reveal a theme (e.g., one cluster dominated by words about *plot/story*, another about *acting/performance*, another about *visuals/animation*). Reviews in a cluster share vocabulary and, loosely, topical focus. Clusters are not perfectly aligned with sentiment (positive/negative are mixed), showing that vocabulary separates *topic* from *opinion*.

8. Conclusion and Future Work

Summary of findings

- Ratings are concentrated at 4-5 stars; the dataset is imbalanced toward positive reviews.
- SnowNLP sentiment correlates positively but only moderately with star ratings; agreement ~ moderate, limited by sarcasm and domain slang.
- TF-IDF / LSA representations give the best binary classification performance; Word2Vec mean is competitive and would improve with pretrained embeddings.
- Clustering on TF-IDF reveals topic-based groups rather than sentiment groups.

Future work

- Use `gensim` Word2Vec or pretrained Chinese embeddings (Tencent AI / w2v.baidu) for richer vectors.
- Try deep models (BERT / RoBERTa Chinese) for classification and sentiment.
- Handle class imbalance (SMOTE, class weights) and evaluate with cross-validation.
- Run NLP steps on the full 1M rows with distributed processing (Dask / Spark).

Reproducibility: Set `random_state=42` everywhere; install `pandas, numpy, matplotlib, seaborn, scikit-learn, scipy, jieba, snownlp, wordcloud`.